

Smart MCQ Solver Challenge

Deep Learning and Generative AI — Project Report

Sagar K Chaudhary | Roll No: 23f2002523

Diploma Level, BS in Data Science and Applications — IIT Madras

1. Abstract

In this competition each question comes with five possible answers and exactly one is correct. The task is to pick the three most likely answers and rank them by confidence. Scoring uses MAP@3, which gives full marks if the correct answer is first, half if second and one third if third.

I built four models plus one diagnostic model. The first was a small network written from scratch. The second was a ready-made model called MiniLM used without any training. The third was DeBERTa, a large pretrained model fine-tuned with LoRA, which was my strongest at 0.9983 validation MAP@3 while updating under 1% of its weights. The fourth was ELECTRA, added late for a reason explained below. I also implemented a retrieval pipeline and an ensembling search, and report why neither was used in the final answer.

The most useful finding was not a better model but an explanation of why no model could do better. My from-scratch model scored a perfect 1.000 on validation, which should be impossible for a model that size on a reasoning task. To understand it I built a model that cannot see the question at all. It also scored 100%, proving the correct answers can be spotted from how they are written. A leaderboard submission built only on that pattern scored 0.75311, confirming it exists in the hidden test data too. I reported this to the course TA, who confirmed it.

This explains why my scores stopped improving. Six setups, from a 644 thousand parameter model to a 400 million parameter one, all scored between 0.749 and 0.753 — about two test rows apart. They were all learning the same shortcut and agreed with each other on over 99% of predictions. The fix came from adding a model that disagreed: ELECTRA matched DeBERTa on only 83% of rows, and blending the two gave my best score, **0.75644**.

2. Introduction

2.1 Problem statement

Every row has a question and five answer choices labelled A to E, one of which is right. I rank them and submit my top three. Because there is only one correct answer, the MAP@3 score per row is 1.000 if it is first, 0.500 if second, 0.333 if third and 0 if absent. The final score is the average over all rows. Random guessing gives about 0.3578 and the qualifying mark was 0.73.

2.2 Project objectives

1. Cross the 0.73 qualifying mark on the Kaggle leaderboard.
2. Build and compare at least three kinds of model — a fine-tuned transformer, a custom network of my own design and a recurrent network — tracking every run on Weights and Biases.
3. Get hands-on practice with Hugging Face, PyTorch, LoRA and a retrieval pipeline.
4. Understand the data well enough to explain why each model behaved as it did, not only report scores.

2.3 Report structure

Section 3 covers the data, the five exploratory analyses I ran, and cleaning. Section 4 covers turning text into numbers. Section 5 explains each model and the investigation that shaped everything after it. Section 6 covers the retrieval experiment and ensembling. Section 7 compares results. Section 8 gives conclusions and Section 9 lists references.

3. Dataset, Exploration and Preprocessing

3.1 What the data looks like

Property	Training set	Test set
Rows	2,000	500
Columns	8 (id, prompt, A–E, answer)	7 (answer hidden)
Missing values	0	0
Average prompt / option length	117 / 165 characters	—
Number of different words	about 2,975	—

Table 3.1 — Size and basic properties. Every column is text; there are no numeric columns.

Since everything is text there was nothing to fill in for missing values, no categories to encode and no features to build by hand. The models learn their own representation, so my only job was converting words into numbers correctly.

Reading one full row was the single most informative step. For a question on Heidegger's view of time, all five options discuss Heidegger and all five discuss time. The wrong answers are not random text — they are deliberately written to look convincing. That observation drove everything that followed.

3.2 Exploratory analysis

I ran five separate analyses, each answering a question that changed a later decision.

3.2.1 Target value analysis

The first check was whether the answer letter is balanced, because if one letter were correct most of the time a model could score well without learning anything. The correct answer is B in 490 rows, C in 459, A in 369, D in 358 and E in 324. The most common option covers only 24.5% of rows, so a constant guesser would reach about 24.5% against a 20% random baseline — there is no easy shortcut in the labels.

The imbalance is still real: B appears about 50% more often than E. That is why I report macro F1 alongside accuracy throughout, since macro F1 weights every option equally and would expose a model that neglects the rarer option E.

3.2.2 Text length analysis

I measured lengths to fix a concrete setting: how many tokens the model is allowed to read. A typical question-and-option pair is about 70 tokens, but the tails matter more than the average. The longest question is 337 characters and the longest option 662, which together come to roughly 250 tokens, so I set the limit to 256 to sit just above the worst case rather than the typical case.

I also checked whether the correct answer is simply the longest option, since that is a common giveaway in multiple choice tests. It is longest in 41% of rows against 20% by chance — a slight tendency, but nowhere near enough to explain a high score.

3.2.3 Vocabulary analysis

For the from-scratch model I had to build my own vocabulary, so I counted every distinct word across all prompts and options. There are only about 2,975 of them — a surprisingly small number, and a real limitation.

It means any word in the test set that never appeared in training becomes a single unknown marker. To that model an unfamiliar technical term and a nonsense string are literally identical inputs. This is a concrete, measurable reason the pretrained models outperform it: their tokenizers break unfamiliar words into smaller familiar pieces, so nothing is ever completely lost.

3.2.4 Prompt template analysis

Reading example rows showed many questions wrapped in the same phrases, such as “Pick the best possible answer:” at the start and “among the listed options.” at the end. One row proves nothing, so I measured the frequency across all 2,500 prompts from both files and found three such endings, each appearing in roughly 400 rows or more.

Text that is identical in hundreds of rows cannot help tell the answers apart — it is the text equivalent of a constant column — and it uses up space that real content could occupy. This analysis is what justified the cleaning step in Section 3.3.

3.2.5 Similarity baseline check

Before building anything heavy I tested the most obvious idea: that the correct answer is the one most similar to the question. I used TF-IDF, which scores words by how often they appear in one text and how rare they are overall, then compared texts by cosine similarity.

It scored 0.1635 accuracy — worse than the 0.20 you get by guessing. So similarity does not merely fail here, it actively points to the wrong answer. The reason is that the wrong options deliberately reuse the question's vocabulary to look convincing while the correct answer often brings in new words, so picking the most similar option systematically favours a distractor.

This test took seconds and redirected the whole project. It told me the task is about judging correctness rather than measuring relatedness, so I stopped trying to improve similarity and moved to models that read the question and the option together.

3.3 Cleaning the text

Acting on Section 3.2.4, I removed the templated phrases with regular expressions and tidied up the spacing. Average question length fell from 117 to 91 characters, a 22% reduction, freeing those tokens for real content. The same cleaning function was run on both the training and test files so the model never sees a different format at prediction time, and I kept the original column untouched so the raw data remained available.

3.4 Data augmentation

I tried one technique: shuffling the order of the five options during training, moving the label along with the correct answer so the target stays right. This was not aimed at improving the score but at testing whether the shortcut described in Section 5.5 depended on the position of the correct answer. It did not help the leaderboard and was not used in the final model, but the negative result was informative and is reported in Table 5.4.

4. Turning Text into Numbers

4.1 Which tokenizer and why

A model cannot read words, only numbers, so text must be split into pieces called tokens and each given a number. My best model uses the tokenizer that ships with DeBERTa-v3, which has about 128,000 pieces. Three reasons:

- 1. It handles unknown words. As Section 3.2.3 showed, a vocabulary built from this dataset has only 2,975 words, so unseen words are lost entirely. The DeBERTa tokenizer breaks unfamiliar words into smaller familiar pieces instead.
- 2. It handles pairs of texts natively, producing a single sequence with a separator between question and option — exactly the format DeBERTa was originally trained on.
- 3. It matches the model. Each token number points to a row in the model's internal table; a mismatched tokenizer would point to the wrong rows and produce nonsense with no error message.

4.2 How it was applied

Each row becomes five pairs — the question with option A, the question with option B and so on. Anything past 256 tokens is cut off, using the limit worked out in Section 3.2.2. For padding I wrote a small function that pads each batch only to the length of its own longest item rather than always to 256, so batches of short texts run faster with no change to the result.

4.3 Comparison of the approaches tried

Approach	Type	Vocabulary	Result
My own word-level tokenizer	split on words	2,975	unknown words lost; scratch model only
TF-IDF	word counting	from the corpus	0.1635 accuracy, worse than guessing
MiniLM WordPiece	sub-word pieces	about 30,000	0.2567 accuracy without training
DeBERTa-v3 SentencePiece	sub-word pieces	about 128,000	0.9983 validation MAP@3 (chosen)
ELECTRA WordPiece	sub-word pieces	about 30,000	0.8900; kept for the final blend

Table 4.1 — Every tokenizing approach I tested.

The pattern is worth noting. Simple word counting did worse than guessing, so surface word overlap actively misleads here. A sub-word vocabulary used without training got above chance but only just. The big jump came only after fine-tuning a model that reads question and option together, which shows the gain came from the model and the training rather than from the tokenizer alone.

5. The Models

Section	Model	Category it covers
5.1	DeBERTa-v3-base + LoRA	Fine-tuned transformer
5.2	Siamese BiGRU dual-encoder	Custom model built from scratch
5.3	Bidirectional GRU (inside 5.2)	Recurrent model
5.4	MiniLM, used without training	Pretrained model, zero-shot
5.6	ELECTRA-base + LoRA	extra model, added for the blend

Table 5.1 — Which model covers which required category. A sixth model was built purely for the investigation in Section 5.5.

5.1 Model 1 — DeBERTa-v3 with LoRA

This is microsoft/deberta-v3-base, a pretrained 12-layer model that has already learned English from a very large amount of text. I attached a small scoring head that gives one score to each of the five options.

Why this model. DeBERTa keeps track of what a word is and where it sits as two separate pieces of information, which helps when the wrong options use almost the same words as the question and what separates them is which statement attaches to which thing. The bigger advantage is that the question and the option go in as one single sequence, so the model can compare their words directly — something a model that encodes the two texts separately, like MiniLM, cannot do.

How I fine-tuned it. Updating all 186 million weights would not fit in the available memory, so I used LoRA. It leaves the original weights frozen and adds small extra matrices next to them, training only those. To give a sense of the saving: a 768 by 768 block of weights holds 589,824 numbers, and LoRA replaces the update to it with two thin matrices holding 24,576 in total, about 96% less.

Setting	Value	Reason
Weights actually trained	1,476,097 of 185,899,010 (0.79%)	the rest stay frozen and keep what they know
LoRA rank / alpha	$r = 16$, $\alpha = 32$	size of the added matrices; $\alpha = 2r$ is usual
Where LoRA attaches	query, key and value projections	these decide what the model pays attention to
Optimizer	AdamW, weight decay 0.01	standard for this kind of model
Learning rate	$2e-4$	higher than usual: the added matrices start from scratch
Warmup / gradient clipping	10% warmup, clip at 1.0	added after training blew up once (Section 5.7)
Epochs / batch size	3 / 4	scores stopped improving after the third pass
Precision	float32	the faster 16-bit mode failed with LoRA (Section 5.7)

Table 5.2 — Training settings and why each was chosen. I did not use quantization, since LoRA alone made the model fit.

Multi-seed bagging

Every training run differs slightly because of random weight initialisation, data shuffling and dropout. Averaging several runs cancels some of that randomness. I trained three times with seeds 42, 123 and 777, scoring 0.9811, 0.9878 and 0.9817 individually, and averaged their predicted probabilities. That moved the leaderboard from 0.75062 for a single run to 0.75187.

5.2 Model 2 — my own network, built from scratch

This model uses nothing pretrained, not even the tokenizer. Following Section 3.2.3 I built the vocabulary by counting words in the training text, keeping slot 0 for padding and slot 1 for unknown words, and turned every text into a fixed list of 80 numbers.

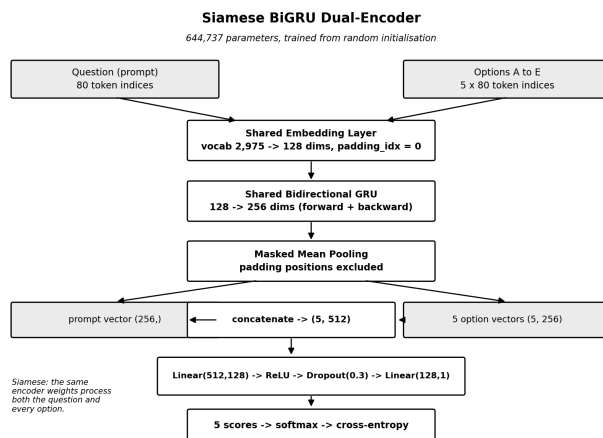


Figure 5.1 — How my from-scratch model is put together.

Why I designed it this way. The same encoder processes both the question and each option. This halves the number of weights and puts both texts into the same kind of representation so they can be compared directly. When averaging the word vectors into one vector per text I skip the padding positions, otherwise short texts get dragged towards zero and cannot be compared fairly with long ones. Instead of just measuring similarity between the two vectors, I join them and pass them through a small network, which lets the model learn how a question and an option relate rather than using a fixed formula — exactly what TF-IDF and MiniLM cannot do. With 644,737 weights against 1,700 rows a perfect score on unseen data should not be possible, which is what made the result worth investigating.

5.3 Model 3 — the recurrent part

The recurrent layer inside the model above is the recurrent model required by the project. It is a bidirectional GRU with a hidden size of 128, producing 256 numbers per word position once both directions are joined.

Why word order matters here. The five options in a question use almost the same words and differ in how those words are arranged — one might say humans exist within time and another that humans are time. If you only count words these look identical, which is exactly why TF-IDF failed. A recurrent layer reads words in order and carries a running memory forward, so it can tell the difference.

Why a GRU. Compared with a plain recurrent network, a GRU has gates that control how much past information carries forward, which stops the signal fading across a long sentence. Compared with an LSTM it has two gates instead of three and about 25% fewer weights, which suits a dataset of only 1,700 rows. Reading in both directions means each word also gets context from what comes after it.

5.4 Model 4 — MiniLM, used without any training

This is the pretrained model required by the project, used exactly as downloaded with no training at all. It turns the question and each option into a vector and ranks the options by how similar their vectors are to the question's.

Its purpose was to answer a specific question raised by Section 3.2.5. TF-IDF failed at matching words, so was the problem word matching, or was it similarity itself? Testing meaning-based similarity without training answers that. It scored 0.4083 MAP@3 and 0.2567 accuracy — better than TF-IDF's 0.1635, so understanding meaning does help, but still close to the 0.3578 and 0.1867 you get from guessing.

The conclusion is that similarity is the wrong framing regardless of how it is measured, because the wrong options are written about the same topic on purpose and are therefore just as similar to the question as the right one. Had I fine-tuned this model instead, it would no longer have been measuring similarity and that conclusion could not have been drawn.

5.5 The main finding: a hidden pattern in the data

My from-scratch model scored a perfect 1.000 on validation. A model with 644,737 weights trained on 1,700 rows should not answer reasoning questions perfectly on data it has never seen, so I treated this as something to explain rather than something to be pleased about.

My guess was that the correct answers are written in a slightly different style from the wrong ones and the model had learned to spot that style. This can be tested cleanly: if the answer really can be identified from the option text alone, a model that never sees the question should still do well; if understanding is genuinely needed it cannot beat 20%. So I built exactly that, using the same encoder and data split so the only thing that changed was whether the question was available.

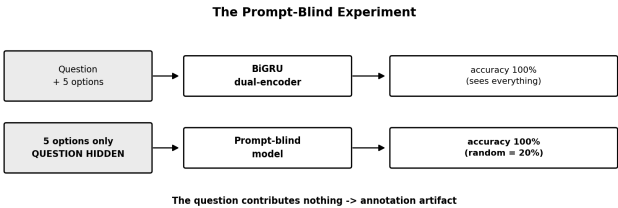


Figure 5.2 — The test. Removing the question completely does not lower the score.

The question-blind model scored 100% on validation without ever seeing a question, against 20% for guessing. I then submitted predictions built only on this pattern and got 0.75311 on the leaderboard, confirming the same pattern exists in the hidden test data.

This is known as an annotation artifact: the correct answers and the wrong ones were produced by different processes, leaving a consistent difference in style. I reported it to the course TA, who confirmed it is a data problem and advised that my final model should not be built on it, since a private leaderboard might not share the pattern.

It is worth being clear that this is not overfitting. In overfitting the model does well on training data and badly on validation data because it has memorised. Here the validation score is also perfect, so nothing was memorised — the model learned a rule that genuinely works on this data, but the rule is “recognise how a correct answer is written” instead of “understand the question”.

How much does my own model rely on it?

I measured this by checking how often DeBERTa and the question-blind model make the same prediction. They agreed on 99.7% of validation rows, about 299 out of 300. That explains the central puzzle of the project: the same model scores 0.9983 on validation but 0.75062 on the leaderboard, because validation comes from the same file as training and is full of the same pattern while the test set is not equally easy to exploit.

Two attempts to reduce the reliance

What I tried	Agreement with question-blind model	Leaderboard
Baseline (3-seed DeBERTa)	99.7%	0.75187
Shuffling the option order	95.7%	0.73773
Label smoothing + cosine schedule	96.0%	0.74937

Table 5.4 — Both attempts worked slightly and both made the leaderboard worse.

Since the test set contains the same pattern, moving away from it costs score. Shuffling failing also tells me the pattern lives in the wording of the options rather than in their position, because shuffling can only disturb position.

5.6 The extra model — ELECTRA and the improvement

After six setups all landed on the same score (Table 7.2), I worked out they were all learning the same thing. Combining models only helps when they make different mistakes, and mine agreed more than 99% of the time. The answer was to bring in a model that disagrees. ELECTRA is trained differently — it learns by spotting which words in a sentence have been swapped out rather than by filling in blanks — so it picks up different patterns. I trained it through the same pipeline, with one change: its internal layer names differ, and giving the wrong ones would attach the LoRA matrices to nothing while training carried on as if fine.

Pair of models	How often they agree
DeBERTa vs question-blind model	99.7%
DeBERTa (3 seeds) vs 5-fold cross-validation	about 99% (same family)
DeBERTa vs ELECTRA	83.0%

Table 5.5 — ELECTRA is the first model in the project that genuinely disagrees.

I measured this agreement before using any submission, which turned a hunch into a prediction I could test. When I checked different blend weights on validation the score did not move at all, because DeBERTa is so confident that its answer wins regardless. So I chose 0.25 by reasoning — a model scoring 0.89 deserves a real say but not the final word — and confirmed it on the leaderboard, where it gave 0.75644 against 0.75187 for DeBERTa alone.

5.7 Problems I ran into

- 1. Training loss exploded. My first run showed a loss of 543 when it should start near 1.61 for a five-option problem. The cause was the new scoring head, which starts with random weights and produces very large corrections in the first few steps. I fixed it by raising the learning rate slowly over the first 10% of training and capping how large any single update can be. Runs then started at about 1.58 as expected, and I used that number as a quick health check on every later run.
- 2. The faster 16-bit training modes did not work. One is unsupported by the GPU I started on; the other clashed with LoRA and threw an error about gradients. I trained in normal 32-bit precision instead, which was affordable precisely because LoRA trains only 0.79% of the weights. On the T4 GPU used for ELECTRA the 16-bit mode worked and was switched on.
- 3. A library conflict on Kaggle stopped LoRA from importing. I removed the conflicting package in the first cell and restarted the kernel.

6. Retrieval and Ensembling Experiments

Two further techniques were implemented in full and then deliberately excluded from the final answer. Both exclusions were measured decisions rather than omissions, so both are reported here.

6.1 Retrieval-augmented generation

The idea behind retrieval augmentation is that a model only knows what is in its weights, so if a fact was never in its training it cannot produce one. Retrieval fetches relevant documents at question time and puts them into the input, letting the model reason over information it never memorised.

I built the standard two-stage pipeline. The first stage uses MiniLM to turn every document into a vector and a FAISS index to find the five nearest ones quickly. The second stage uses a cross-encoder, which reads the question and each candidate together and scores them far more accurately, but is too slow to run over a whole collection — hence using the fast stage to narrow it down first.

Retrieval quality	Value	What it means
Hit rate on training rows	83%	meaningless — each row's own answer is in the collection
Hit rate on validation rows	68%	the real figure, and even this reflects similar-looking text

Table 6.1 — Measured retrieval quality.

The competition provides no external source of information, so the collection had to be built from the training answers themselves. A test question's answer is simply not in there, which means retrieval can only return documents about other questions. Since a model trusts whatever context it is given, irrelevant context is actively harmful rather than merely useless. I therefore kept the pipeline as a study of the technique and left it out of the final answer.

6.2 Ensembling search

I ran a weight search over combinations of my first three models on the validation set. It returned a perfect score for several combinations, which is a symptom rather than a result: validation is full of the shortcut described in Section 5.5, so every reasonable model scores above 0.99 there and the search cannot tell them apart.

More fundamentally, combining models only helps when they make different mistakes. Measuring the agreement between my models showed over 99%, which explained immediately why averaging them changed nothing. That measurement is what led to Section 5.6 and the ELECTRA blend, which is the version of ensembling that actually worked.

7. Results

7.1 The measures I used

MAP@3 is the competition's own measure and my main one. I also report accuracy, which counts only whether my first choice was right, and macro F1, which treats all five options equally and would reveal a model ignoring the less common ones. All three come from a single function that I tested against an example computed by hand, so every number here is comparable with every other.

7.2 Training curves

All the curves and graphs can be found on the wandB website.

7.3 Comparison of all models

Model	MAP@3	Accuracy	Macro F1	Time
Random guessing	0.3578	0.1867	0.1859	—
TF-IDF similarity	—	0.1635	—	seconds
My own BiGRU (from scratch)	1.0000	1.0000	1.0000	~1 min
MiniLM (pretrained, no training)	0.4083	0.2567	0.2515	none
DeBERTa-v3 + LoRA	0.9983	0.9967	0.9966	~6 min
DeBERTa with option shuffling	0.9667	0.9567	0.9543	~6 min
DeBERTa averaged over 3 seeds	0.9872	0.9767	0.9756	~18 min
ELECTRA + LoRA (2 seeds)	0.8900	0.8267	0.8279	~8 min
Final blend (0.75 DeBERTa + 0.25 ELECTRA)	0.9872	0.9767	0.9756	—
Question-blind model (test only)	1.0000	1.0000	1.0000	~1 min

Table 7.1 — All models on the same held-out validation data. Read every number with the Section 5.5 warning: validation contains the same shortcut as training, so these scores flatter every model. The question-blind row is included on purpose — any model scoring close to it is probably using the same shortcut.

What the comparison shows. My from-scratch model trains in a minute but its advantage is entirely the shortcut. DeBERTa-large has three times as many weights yet scored 0.74937 against 0.75062, so size was not the limiting factor. MiniLM needed no training and reached 0.4083, while six minutes of fine-tuning took DeBERTa to 0.9983 — showing how much is gained by training on the actual task.

7.4 Kaggle results

What I submitted	Score	Outcome
Question-blind model (shortcut only)	0.75311	not used, on the TA's advice
DeBERTa-base + LoRA, single run	0.75062	baseline
DeBERTa-base + LoRA, 3 seeds averaged	0.75187	best single-family result
DeBERTa-large + LoRA	0.74937	three times bigger, no gain
5-fold cross-validation	0.74930	all 2,000 rows used, no gain
Label smoothing + cosine schedule	0.74937	less shortcut reliance, lower score
Shuffling the option order	0.73773	same pattern, larger drop
DeBERTa + ELECTRA blend, w = 0.25	0.75644	FINAL SUBMISSION
DeBERTa + ELECTRA blend, w = 0.40	0.75602	the same within noise
Adding RoBERTa as a third model	0.74896	a third model did not help

Table 7.2 — Every submission I made.

Final leaderboard score: 0.75644 (qualifying mark 0.73; random guessing 0.3578).

Are these differences real? The test set has 500 rows, so getting one more right changes the score by 0.002 — the smallest difference the score can express. Six setups ranging from 644 thousand to 400 million weights all landed between 0.749 and 0.753, a gap of 0.004 or about two rows. A model six hundred times larger making no measurable difference is strong evidence that size was never the problem. The only change beyond that range was blending two different families, taking the score from 0.75187 to 0.75644, roughly 2.3 rows — and that was also the only change bringing in a model that disagreed with the others, exactly as predicted before submitting it.

7.5 Looking at the mistakes

Measure	Value	What it means
Average confidence when right	0.9350	the model is sure when correct
Average confidence when wrong	0.3643	genuinely unsure when it errs
Correct answer in the top three	100.0%	nothing lost by submitting only three
Correct answer ranked first	99.7%	the remaining loss is one second-place answer

Table 7.3 — Mistake analysis on 300 validation rows.

- The mistakes are spread across all five letters rather than piling onto one. Even though B and C appear more often in training the model does not favour them, so it has not picked up a bias towards common answers.
- The gap between confidence when right and wrong matters: it shows the confidence scores mean something, it explains why the correct answer usually still appears second when the first guess is wrong, and it is what makes averaging several models sensible.
- Reading the actual mistakes, the answer picked is on the same topic and reads sensibly — in one question about crystallographic point groups it answered 27 when the correct value is 32, with confidence only 0.364. So the remaining errors are genuine gaps in knowledge, not data-handling problems. The spread of first choices in the final submission (A=91, B=115, C=116, D=92, E=86) confirms predictions are not collapsing onto one option.

8. Deployment

The trained model is deployed as a web application so that anyone can enter a question and five options and see the ranked prediction, rather than having to run the notebook.

8.1 What was deployed

Component	Where
Web application	Streamlit Community Cloud
Model weights	HuggingFace Hub
Source code	GitHub
Model deployed	DeBERTa-v3 with LoRA (the stronger half of the final blend)

Table 8.1 — Deployment components. Links are given at the end of this section.

The application takes a question and five options, tokenizes each option paired with the question exactly as during training, runs the model, and returns the three most likely answers in ranked order along with the confidence for all five. It also shows a warning when the top confidence is low, since the error analysis in Section 7.5 showed the model averages 0.94 confidence when correct and 0.36 when wrong — so a low score genuinely means uncertainty rather than a random output.

8.2 Preparing the model

Two steps were needed before the model could be served. First the LoRA adapters were merged back into the base weights with `merge_and_unload()`. Adapters are stored separately during training, so without merging the application would need the PEFT library and would have to reassemble the model at every start. Merging produces one ordinary model that loads with a single call.

Second, the merged model was pushed to the HuggingFace Hub rather than committed alongside the code. GitHub rejects files over 100 MB and the merged model is 870 MB, so the weights and the application have to live in different places. The application downloads them at first run and caches them afterwards.

8.3 The memory constraint

The main engineering problem was memory. Streamlit's free tier allows 1 GB of RAM, while the model alone is 870 MB in float32 before counting PyTorch's own overhead. Three measures were applied:

1. Loading with `low_cpu_mem_usage=True`, so the weights stream in shard by shard. Without it PyTorch first builds an empty model and then fills it, which briefly holds two full copies in memory — precisely at the moment when memory is tightest.
2. Quantizing the linear layers to int8 at load time. This reduces the footprint substantially and also speeds up inference on CPU, at a small accuracy cost that does not matter for a demonstration.
3. Caching the loaded model with Streamlit's `cache_resource` decorator. Streamlit re-executes the whole script on every interaction, so without caching the model would reload on every button press.

Pinning dependency versions also proved necessary. The host defaulted to a Python and PyTorch combination newer than the one used for training, which pulled in a major version of the transformers library with breaking changes, along with several hundred megabytes of CUDA packages on a machine with no GPU. Fixing the Python version and pinning each package to the versions used in the notebook resolved both problems.

8.4 What is deployed versus what was submitted

The competition submission is a blend of DeBERTa and ELECTRA. The deployed demonstration runs DeBERTa alone, because loading two transformers is not possible within the available memory. The blend contributes 0.0046 MAP@3, roughly two test rows, at the cost of running a second full model — a trade worth making for a competition score but not for a served application. This is a normal deployment decision: the marginal accuracy has to justify the marginal cost.

Link	URL
Live demo	https://mcq-solver-app-deep-learning.streamlit.app/
Model	huggingface.co/Tokyo0412/mcq-solver-deberta
Source	github.com/Perceptron04/mcq-solver-app

Table 8.2 — Deployment links.

9. Conclusion and Future Work

9.1 What I learned

1. Being similar is not the same as being correct. Word-matching scored 0.1635 and meaning-matching 0.2567, both at or below guessing, because the wrong options are written about the same topic on purpose. Finding this out in the first hour saved a lot of wasted effort.
2. A result that looks too good is a reason to check, not to celebrate. A perfect validation score from a very small model led directly to the main finding of this project.
3. A model taking a shortcut is a different problem from a model memorising, and telling them apart needs an experiment rather than a number. Building a model that physically cannot see the question gave me evidence no score could have.
4. When validation cannot be trusted the leaderboard is the only honest signal, and differences must be judged against what the score can measure — with 500 test rows anything under 0.004 is about two rows and proves nothing.

5. Combining models only helps if they are genuinely different. Averaging runs of the same model does nothing when they agree 99% of the time. Measuring that agreement first turned guesswork into a prediction, and ELECTRA at 83% was the change that worked.
6. Training a small fraction of a large model works remarkably well: updating 0.79% of the weights reached 0.9983 on validation, and the low memory cost let me train in full precision when the faster modes failed.
7. Implementing a technique and then not using it is a legitimate result. Both the retrieval pipeline and the ensembling search were built in full, measured, and excluded for reasons I can defend.

9.2 Problems and how I solved them

Problem	How I solved it
Training loss exploded to 543	traced to the new scoring head starting from random weights; fixed with slow warmup and a cap on update size
16-bit modes failed; a library conflict broke LoRA	trained in 32-bit, affordable thanks to LoRA; removed the conflicting package first
Validation scores could not be trusted	the question-blind experiment measured how bad it was; after that every decision was checked on the leaderboard
Score stuck across six setups	found the cause by measuring how much the models agreed; fixed by adding a model trained a different way
Retrieval returned irrelevant documents	measured the hit rate, understood the collection had no useful content, and excluded the pipeline

Table 9.1 — The main problems and what I did about them.

9.3 What I would try next

- Train the model on some examples where the question has been removed and the answer is random, so it is directly penalised for answering without reading the question. Shuffling the options only changed their position; this would target the actual cause.
- Pair each question with wrong options taken from other questions, breaking the link between writing style and correctness in the training data itself.
- Search more systematically for models that disagree, since that was the only thing that worked — choosing them by measured disagreement rather than by how well they score alone. Combining models by rank instead of by score would also stop one very confident model from dominating.
- Find a real external source of information, which would make the retrieval pipeline in Section 6.1 genuinely useful rather than a study of the technique.